

Proyecto final

Sistemas Empotrados Distribuidos



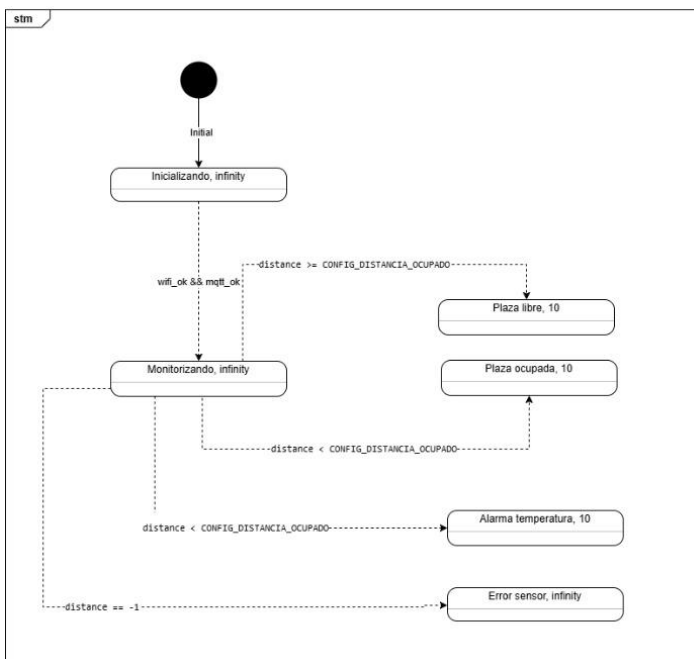
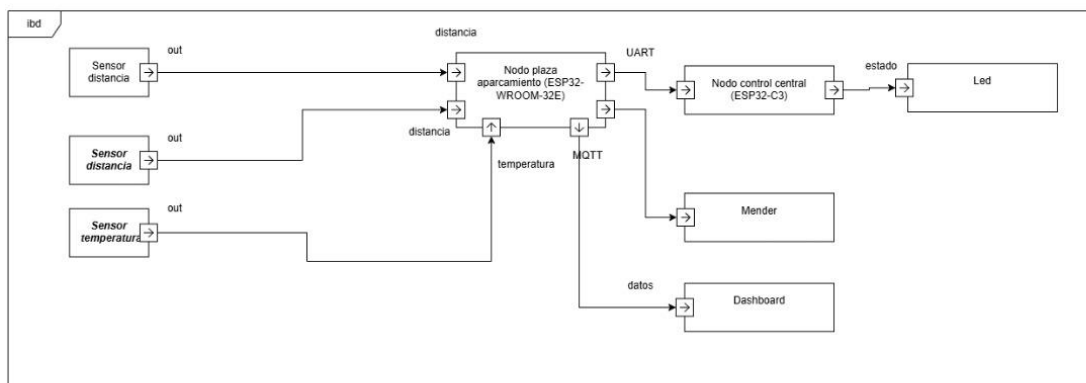
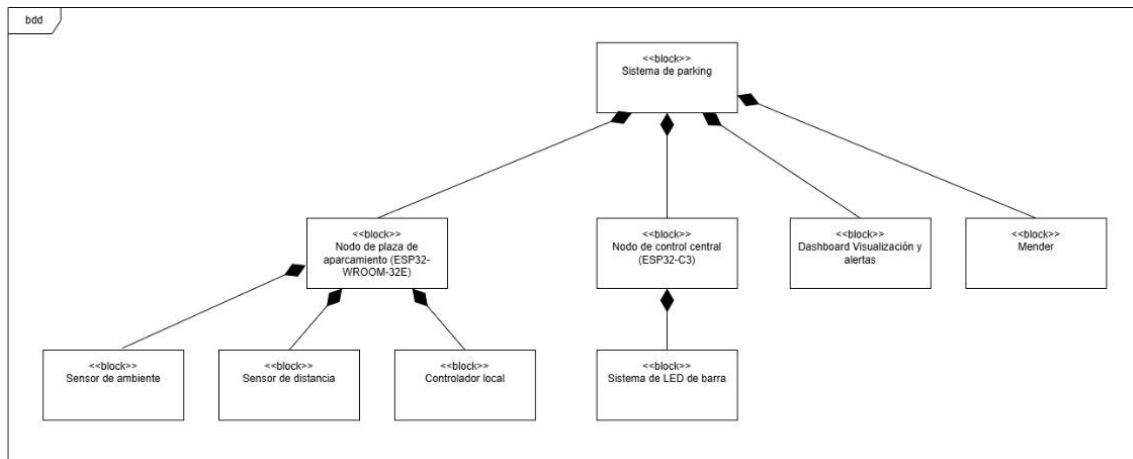
Natalia Paloma La Rosa Montero

Iremar Luhetsy Rivas Álvarez

• Modelado SysML	1
• Análisis de resiliencia	1
1. Gestión de pérdida de conexión Wi-Fi	1
Detección de conexión	1
Recuperación tras fallo	1
2. Resiliencia MQTT	1
Last Will and Testament (LWT)	1
Keepalive MQTT	1
3. Gestión de errores de sensores ultrasónicos	1
Control de timeout	1
Sustitución por última medida válida	1
Indicadores de error	1
4. Resiliencia frente a errores de temperatura	1
5. Resiliencia mediante FreeRTOS	1
Beneficios	1
Aislamiento de fallos	1
Comunicación segura	1
Ejecución periódica estable	1
6. Resiliencia UART entre dispositivos	1
7. Recuperación ante reinicios	1
Persistencia OTA	1
Reconexión automática	1
• Sistema OTA	1
Arquitectura OTA implementada	1
Funcionamiento general	1
1. Generación del firmware OTA	1
2. Identificación única del dispositivo	1
3. Inicialización del cliente Mender	1
4. Activación del servicio OTA	1
5. Descarga e instalación OTA	1
6. Verificación tras actualización	1

7. Health Check automático	1
8. Confirmación del firmware	1
9. Rollback automático	1
Gestión de estados OTA	1
Estado del despliegue	1
Autenticación	1

- **Modelado SysML:** Diagramas estructurales y, opcionalmente, de comportamiento.



- **Análisis de resiliencia:** Explicad cómo gestiona el sistema los fallos (pérdida de Wi-Fi, reinicios, errores de sensores).

1. Gestión de pérdida de conexión Wi-Fi

El sistema utiliza el modo estación (WIFI_MODE_STA) del ESP-IDF para conectarse a la red inalámbrica.

Detección de conexión

La función:

```
perform_health_check()
```

comprueba si el ESP32 sigue asociado al punto de acceso utilizando:

```
esp_wifi_sta_get_ap_info()
```

Si esta función falla el sistema detecta pérdida de conectividad y se registra el error mediante ESP_LOGE.

Recuperación tras fallo

Cuando el dispositivo arranca tras una actualización OTA:

```
check_and_commit_ota()
```

Verifica el estado de la nueva imagen y la disponibilidad de la red Wi-Fi.

Si la comprobación falla el firmware se marca como inválido y el sistema ejecuta rollback automático.

```
esp_ota_mark_app_invalid_rollback_and_reboot();
```

Esto evita dejar el dispositivo inutilizable tras una actualización defectuosa.

2. Resiliencia MQTT

La comunicación MQTT está diseñada para minimizar pérdidas de estado.

Last Will and Testament (LWT)

El cliente MQTT configura un mensaje “Offline”:

`.session.last_will`

Si el ESP32:

- pierde alimentación,
- se reinicia inesperadamente,
- o pierde conexión,

el broker publica automáticamente:

Offline

en:

`sed/G02/status`

Cuando el dispositivo vuelve a conectarse publica:

Online

Esto permite monitorizar disponibilidad en tiempo real.

Keepalive MQTT

El sistema usa:

`.session.keepalive = 10`

Con ello el broker detecta rápidamente clientes caídos, se reducen sesiones zombie y mejora la recuperación automática.

3. Gestión de errores de sensores ultrasónicos

Los sensores HC-SR04 pueden fallar por ecos inválidos, ausencia de obstáculos, ruido eléctrico y tiempos de espera excesivos.

Control de timeout

La función:

`medir_distancia()`

implementa límites temporales:

```
if (esp_timer_get_time() - start > 30000)
    return -1;
```

Si no llega señal se devuelve -1, evitando bloqueos infinitos.

Sustitución por última medida válida

En la tarea de procesamiento:

```
sensor_processing_task()
```

si:

- la distancia es inválida,
- o supera 160 cm,

el sistema reutiliza la última medida correcta:

```
received_data.distance1 = ultima_dist1;
```

Esto aporta estabilidad, reducción de falsas alarmas y continuidad operativa.

Indicadores de error

El JSON transmitido incluye:

```
"error1":1,
"error2":1
```

Permitiendo diagnóstico remoto, detección de sensores defectuosos y análisis desde Node-RED o MQTT.

4. Resiliencia frente a errores de temperatura

El sensor Si7021 devuelve códigos ESP_OK.

Solo si ambas lecturas son válidas:

```
if (res1 == ESP_OK && res2 == ESP_OK)
```

los datos se envían a la cola FreeRTOS.

Esto evita propagar datos corruptos, publicar medidas inválidas y activar alarmas falsas.

5. Resiliencia mediante FreeRTOS

El sistema desacopla adquisición y procesamiento usando:

- tareas (Tasks),
- colas (Queues),
- temporizadores (esp_timer).

Beneficios

Aislamiento de fallos

Si una lectura falla no bloquea el sistema completo.

Comunicación segura

La cola:

```
xQueueCreate()
```

protege el intercambio de datos entre tareas.

Ejecución periódica estable

El temporizador hardware mantiene adquisición constante incluso bajo carga.

6. Resiliencia UART entre dispositivos

La comunicación UART entre ESP32 y ESP32-C3 utiliza mensajes JSON completos e incluye validación implícita mediante búsqueda de patrones.

El ESP32-C3 solo activa el LED si detecta simultáneamente:

"p1":1

"p2":1

Si el mensaje llega incompleto no se activa el LED, evitando estados incorrectos.

7. Recuperación ante reinicios

El sistema soporta: reinicios manuales, watchdog, reinicios OTA y pérdidas de alimentación.

Persistencia OTA

Gracias a Mender la partición válida se conserva y el rollback protege versiones estables.

Reconexión automática

Tras reinicio Wi-Fi se reinicializa, MQTT vuelve a conectarse y Mender reactiva polling OTA.

- **Sistema OTA:** Descripción del proceso de actualización remota implementado.

Arquitectura OTA implementada

El sistema está compuesto por:

ESP32: Dispositivo a actualizar

ESP-IDF OTA: Gestión de particiones y firmware

Mender Client: Cliente OTA residente en el ESP32

Servidor Mender: Distribución de actualizaciones

Artefacto .mender: Paquete del firmware

Funcionamiento general

El proceso OTA sigue este flujo:

1. Compilar nuevo firmware
2. Generar artefacto Mender
3. Subir artefacto al servidor Mender
4. ESP32 consulta nuevas versiones
5. Descarga firmware OTA
6. Instala nueva partición
7. Reinicia el sistema
8. Ejecuta autodiagnóstico
9. Valida o revierte firmware

1. Generación del firmware OTA

Tras compilar el proyecto:

```
idf.py build
```

se genera el binario:

```
build/parking_mqtt.bin
```

Posteriormente se crea el artefacto OTA:

```
mender-artifact write rootfs-image \  
--compression none \  
--device-type esp32 \  
--artifact-name 1.0.1.i \  
--file build/parking_mqtt.bin \  
--output-path a7.2_1.0.1.i.mender
```

2. Identificación única del dispositivo

Cada ESP32 se identifica mediante su dirección MAC:

```
esp_read_mac(mac, ESP_MAC_WIFI_STA);
```

La MAC se registra como identidad Mender:

```
mender_identity[0].name = "mac";
```

Esto permite distinguir dispositivos, desplegar actualizaciones selectivas y gestionar inventario OTA.

3. Inicialización del cliente Mender

El cliente OTA se configura mediante:

```
mender_client_config_t mender_config;
```

Parámetros principales:

Host: <https://eu.hosted.mender.io/ui>

tenant_token: Copiado de mender

4. Activación del servicio OTA

El cliente se ejecuta en segundo plano:

```
mender_client_activate();
```

A partir de ese momento el ESP32 consulta periódicamente el servidor, detecta nuevas versiones y descarga actualizaciones automáticamente.

5. Descarga e instalación OTA

Cuando existe una nueva versión:

1. Mender descarga el artefacto .mender.
2. El firmware se escribe en una partición OTA secundaria.

3. El bootloader cambia la partición de arranque.
4. El ESP32 reinicia automáticamente.

La función callback:

```
mender_restart_cb()
```

ejecuta:

```
esp_restart();
```

6. Verificación tras actualización

Después del reinicio, el firmware entra en modo:

```
ESP_OTA_IMG_PENDING_VERIFY
```

En esta fase el nuevo firmware aún no es definitivo y debe demostrar que funciona correctamente.

7. Health Check automático

La función:

```
check_and_commit_ota()
```

realiza una comprobación de salud.

Actualmente verifica conectividad Wi-Fi y acceso al punto de acceso.

```
perform_health_check()
```

8. Confirmación del firmware

Si el sistema funciona correctamente:

```
esp_ota_mark_app_valid_cancel_rollback();
```

El firmware queda validado, se convierte en versión estable y se cancela cualquier rollback.

9. Rollback automático

Si el diagnóstico falla:

```
esp_ota_mark_app_invalid_rollback_and_reboot();
```

Entonces la imagen se marca como inválida, el ESP32 reinicia y vuelve automáticamente a la versión anterior.

Gestión de estados OTA

El sistema informa del estado OTA mediante callbacks:

Estado del despliegue

```
mender_deployment_status_cb()
```

Permite registrar: descarga, instalación, éxito o fallo.

Autenticación

```
mender_auth_success_cb()
```

Confirma autenticación correcta con el servidor y activación del dispositivo.